Aula 04 – Caminho de dados e controle

Prof. João Fernando Mari
joaofmari.github.io

Roteiro

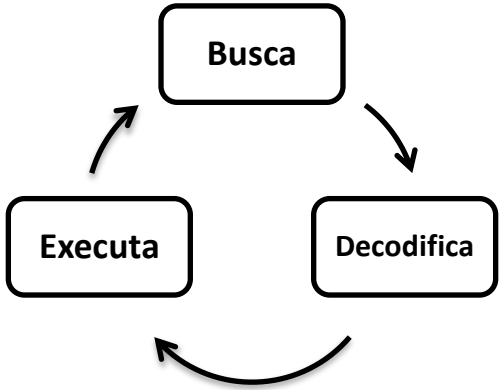
- Introdução
- Sinopse da implementação
- Temporização (*clocking*)
- CONSTRUINDO O CAMINHO DE DADOS
- Busca das instruções
- Banco de registradores
- Instruções lógicas e aritméticas
- Instruções de load e store
- A instrução beq (bne)
- Operações lógicas/aritméticas e de acesso à memória
- Integrando o caminho de dados – MIPS Monociclo
- Simuladores

Introdução

- O desempenho de um programa depende:
  - Número de instruções (depende do conjunto de instruções)
  - Velocidade de clock (depende da implementação)
  - **CPI - Número de ciclos gastos por instrução (depende da implementação)**
- Implementação do MIPS simplificada:
  - Instruções de referência à memória: **`lw`**, **`sw`**
  - Instruções lógicas e aritméticas: **`add`**, **`sub`**, **`and`**, **`or`**, **`slt`**
  - Instruções de controle de fluxo (saltos): **`beq`**, **`j`**

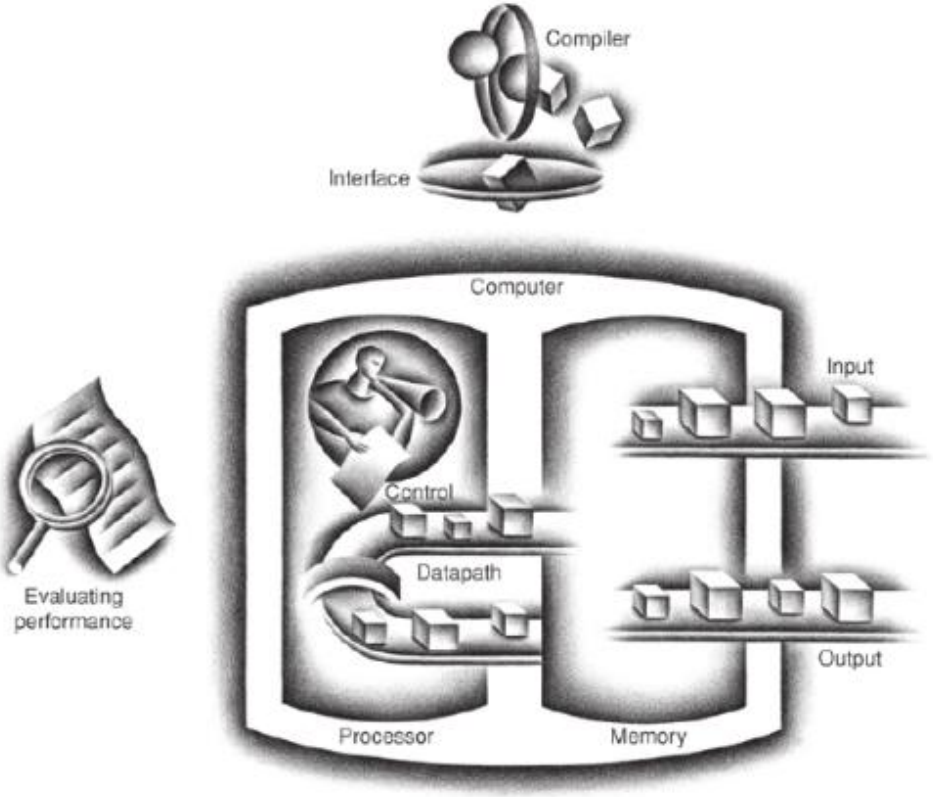
Introdução

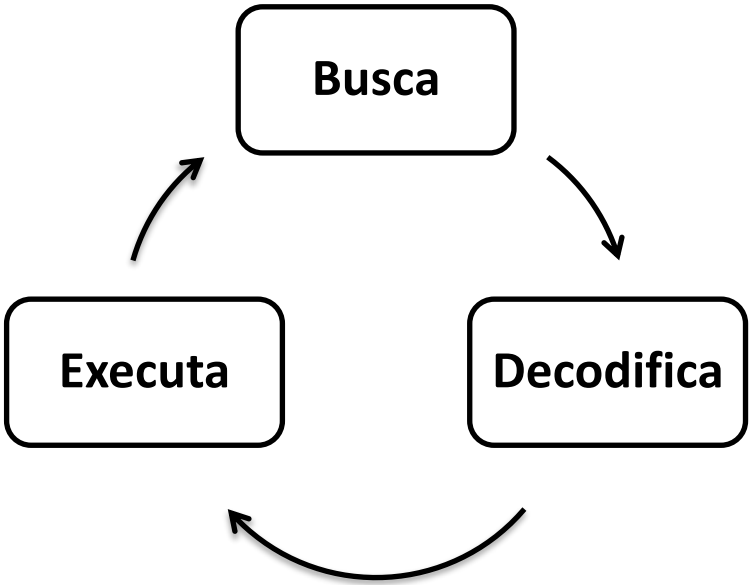
- Implementação geral
  - **BUSCA** a instrução no endereço e memória apontado pelo contador de programa (PC).
    - Atualiza o PC (PC=PC+4)
  - **DECODIFICA** a instrução.
    - Lê os registradores
  - **EXECUTA** a instrução
- Todas as instruções (exceto j) usam a ULA após a leitura dos registradores.



Introdução

- O ciclo básico de busca e execução de instruções.



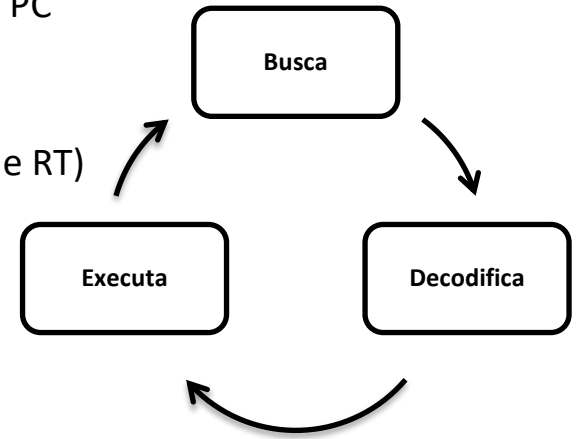


Introdução

- Implementação simplificada
  - Diretrizes:
    - Torne o caso comum mais rápido
- A simplicidade favorece a regularidade
  - As demais instruções podem ser implementadas com princípios semelhantes
  - A arquitetura do conjunto de instruções influencia os aspectos de implementação:
    - Instruções mais simples necessitam de uma implementação também mais simples
- Memória
  - Inicialmente as **memórias de instruções e memória de dados separadas**

Sinopse da implementação

- Todas as instruções possuem os dois primeiros passos idênticos:
- **BUSCA** na memória de instruções a instrução no endereço armazenado em PC
- **DECODIFICA** a instrução na unidade de controle
  - Leitura dois registradores RS e RT
    - A maioria das instruções exige a leitura de dois registradores (RS e RT)
    - Load word exige a leitura de apenas um registrador (RS)
    - Jump não precisa ler nenhum registrador
- **EXECUTA:**
  - Todas as instruções utilizam a UAL após a leitura dos registradores:
    - Instruções de acesso à memória: cálculo do endereço (soma)
    - Instruções lógicas e aritméticas: executam a operação indicada pelo campo FUNCT
    - Desvios condicionais: efetuam uma comparação (subtração)
  - Após usar a ULA, as ações necessárias diferem
    - Acesso à memória:
      - Store word escreve dado na memória
      - Load word lê dado da memória e grava no registrador
    - Instrução aritmética: escreve dado no registrador



Sinopse da implementação

Operação aritmética ou lógica:
- add $s1, $t0, $s2 # RD = RS + RT

Acesso à memória:
- lw $t1, 16($s1) # RT = Memória(RS + offset)
- sw $t1, 16($s1) # Memória(RS + offset) = RT

Branch:
- beq $t1, $t2, L # Se $t1 == $t2, PC = PC + 4 + num. inst. até L

Jump:
- J Label # PC = PC + 4 + num. inst. até Label

PC
Memória de instruções

BUSCA

Lógica/aritmética
Load word
Store word
Branch
Jump

Unidade de controle
Lê dois registradores – RS e RT

DECODIFICA

Lógica/aritmética
Load word
Store word
Branch
Jump

	31 – 26	25 – 21	20 – 16	15 – 11	10 – 6	5 – 0
R	**opcode**	**rs**	**rt**	**rd**	**shamt**	**funct**

	31 – 26	25 – 21	20 – 16	15 – 0
I	**opcode**	**rs**	**rt**	**endereço/imediato**

	31 – 26	25 – 0
J	**opcode**	**endereço**

EXECUTA

Operação aritmética/lógica. *Grava em RD*

ULA → Grava registrador

Lógica/aritmética — Lógica/aritmética

Soma: RS + offset *Lê Memória(RS + offset)* *Grava em RT*

ULA → Lê Memória → Grava registrador

Load word
Store word — Load word — Load word

Subtrai: RS – RT *Grave RT em Memória(RS + offset)*

ULA → Grava memória

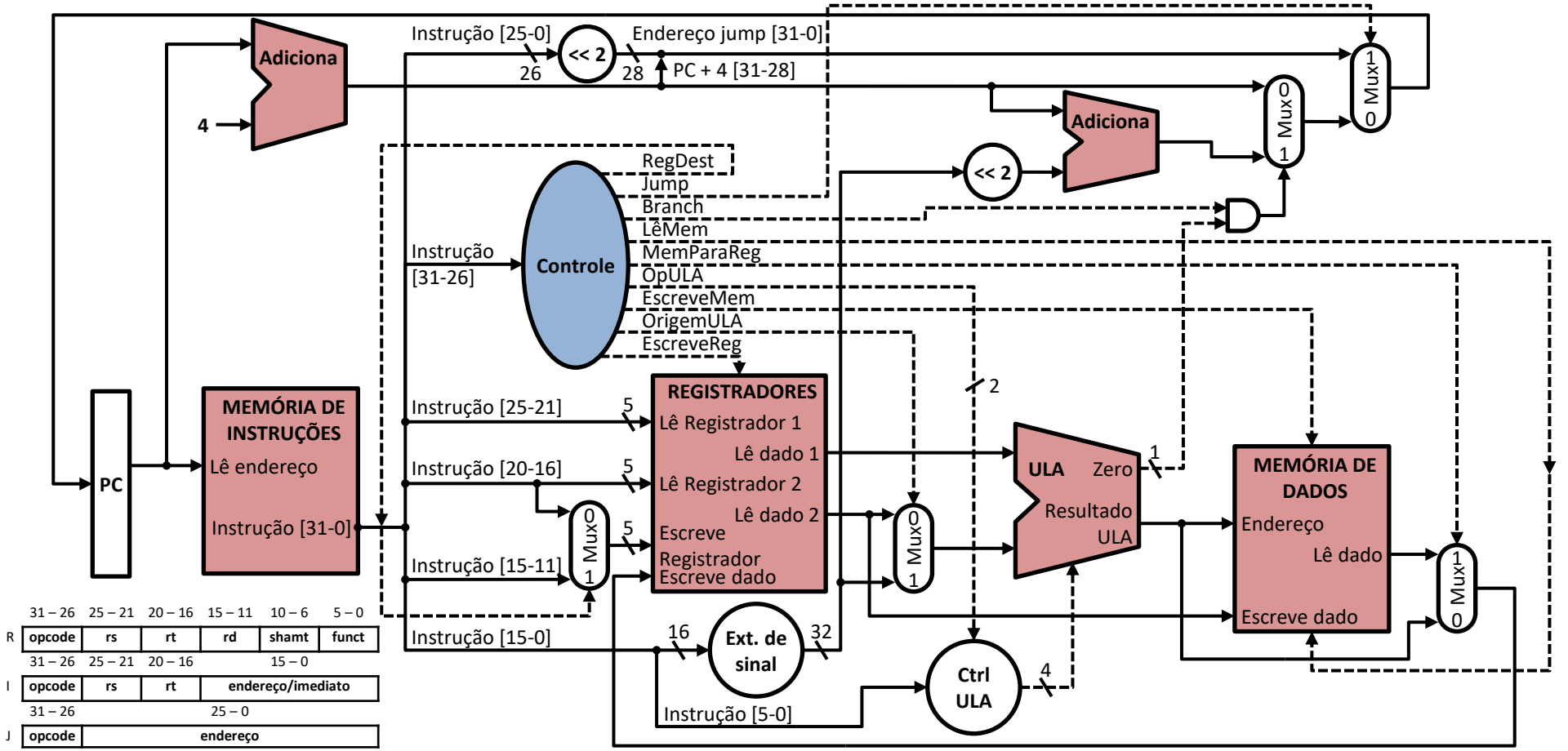
Branch — Store word

Somente se RS – RT == 0, para beq.

Sobrescreve PC → Sobrescreve PC

Jump — Branch

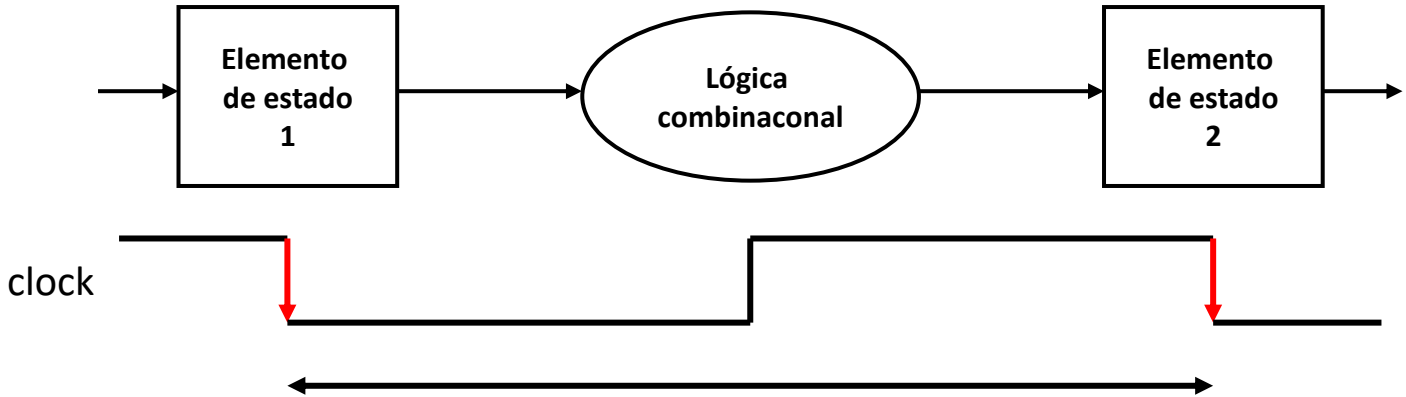
Sinopse da implementação



Temporização (*clocking*)

- Sincroniza quando os dados podem ser lidos e quando podem ser escritos
  - Evita que um dado seja lido ao mesmo tempo que é escrito
    - A leitura pode retornar o valor antigo, o valor recente ou uma combinação dos dois
- Sincronização acionada por transição
  - Quaisquer valores armazenados em um elemento lógico sequencial são atualizados apenas em uma transição de clock
  - Não permite que um elemento de estado seja lido e escrito no mesmo ciclo de clock
    - Não cria uma disputa que poderia levar a valores de dados indeterminados.
  - **O período de clock necessita ser longo o suficiente para que os valores de saída estabilizem.**
- Apenas os elementos de estado podem armazenar valores de dados
  - Coleções de lógica combinatória precisa ter suas entradas vindas de um conjunto de elementos de estado
  - Suas saídas são escritas em um conjunto de elementos de estado.

Temporização (*clocking*)

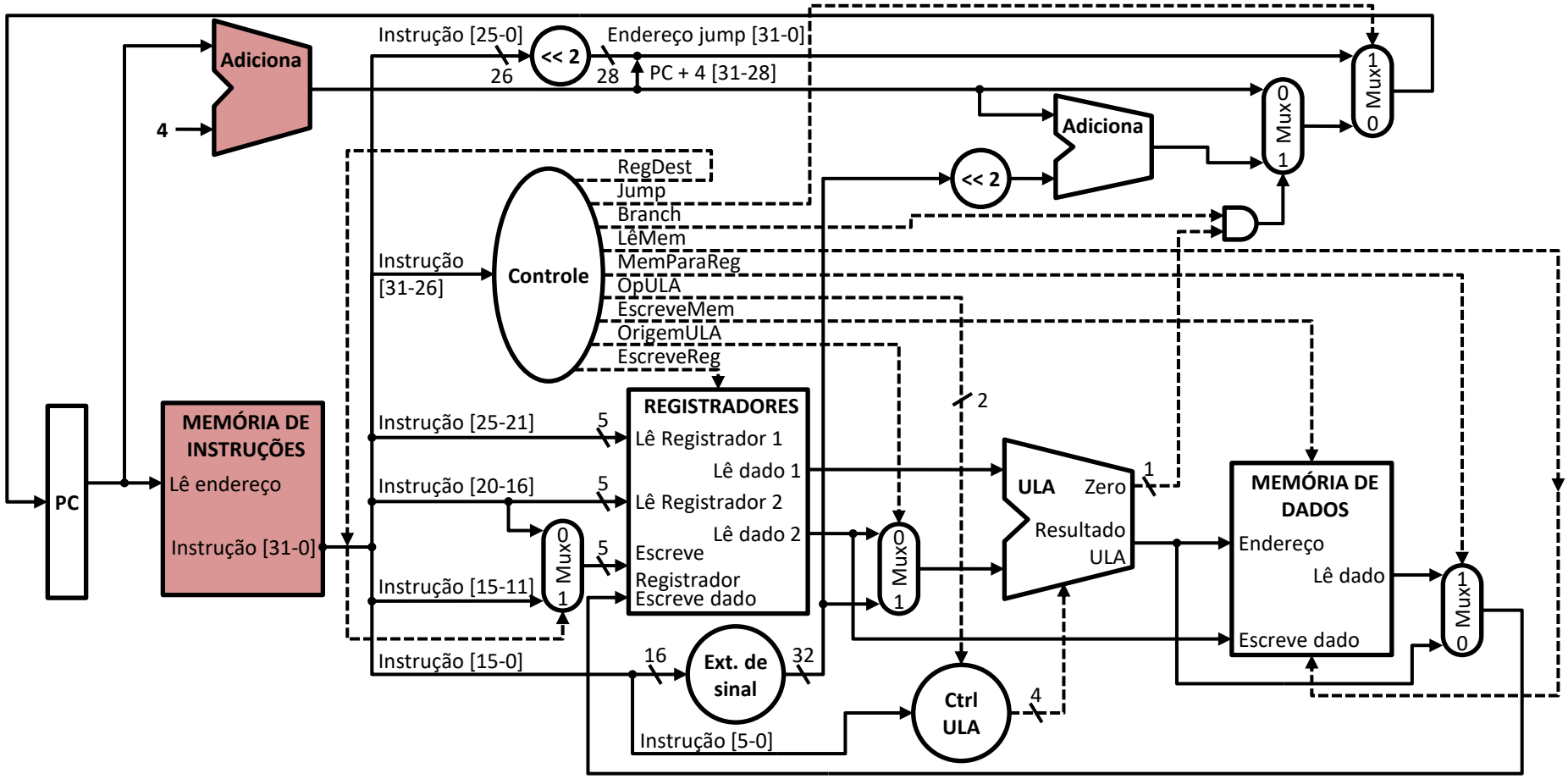


CONSTRUINDO O CAMINHO DE DADOS

Busca das instruções

- Lê a instrução da memória de instruções
  - Atualiza o PC para guardar o endereço da próxima instrução
- PC é atualizado a cada ciclo de clock
  - Por isso não precisa de um sinal de controle de escrita
- A memória de instruções é lida a cada ciclo
  - Por isso não precisa de um sinal de controle de leitura

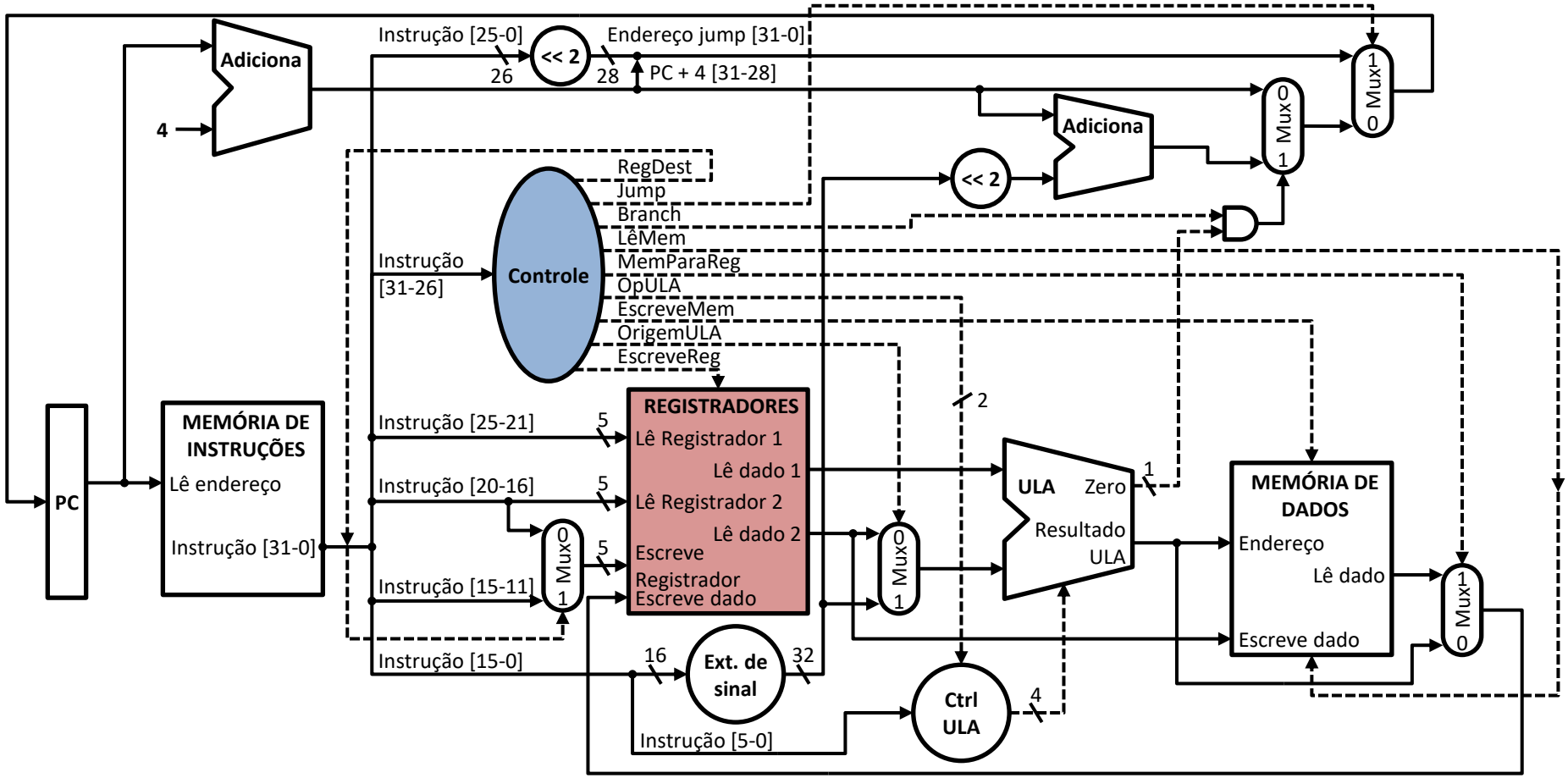
Busca das instruções



Banco de registradores

- Instruções lógicas e aritméticas estão em formato R
  - Exceto pelas instruções imediatas
- Instruções de formato R possuem três operandos de registrador
  - Lê duas palavras de dados do banco de registradores
  - Escrever uma palavra de dados no banco de registradores.
  - 3 entradas de 5 bits (32 registradores):
    - 2 entradas com endereço dos registradores lidos
    - 1 entrada com endereço do registrador escrito
  - 2 saídas de 32 bits: operandos para a UAL
  - 1 entrada de 32 bits: escrita do resultado
- As escritas são controladas pelo sinal de controle de escrita
  - Precisa estar ativo para que um escrita ocorra na transição do clock.

Decodificação e banco de registradores

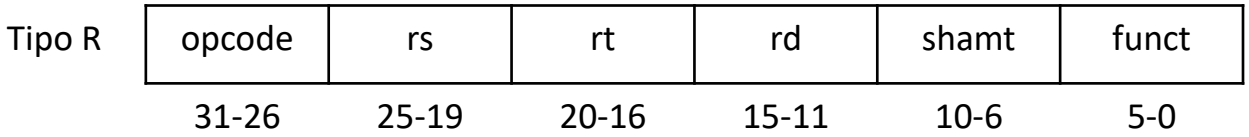


Operações lógicas/aritméticas e de acesso à memória

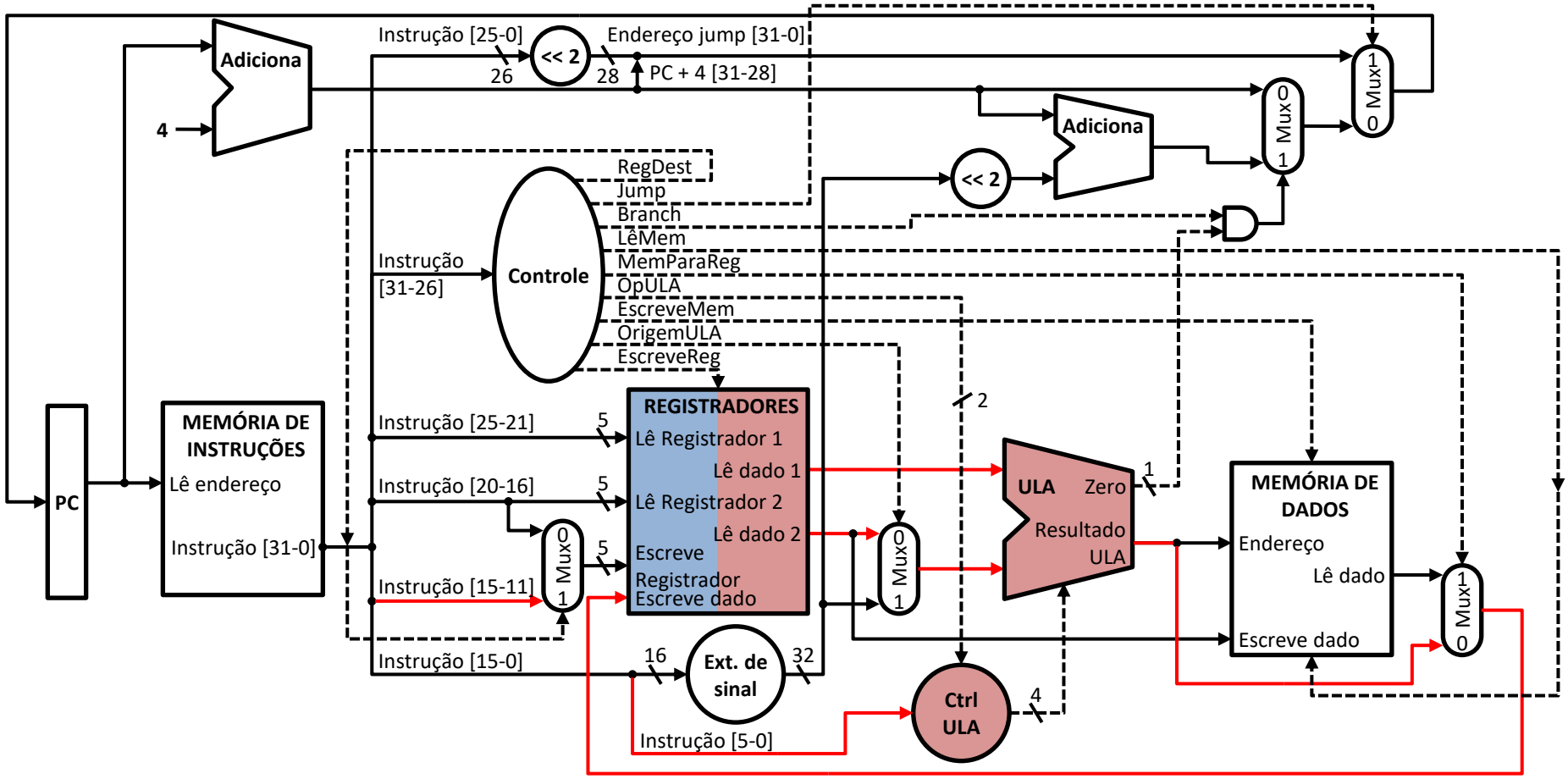
- As instruções lógicas e aritméticas usam a ALU com as entradas vindas de dois registradores;
- As instruções de acesso à memória também podem usar a ALU para fazer o cálculo do endereço
  - Nesse caso, a segunda entrada é o campo offset de 16 bits com o sinal estendido da instrução
- O valor armazenado em um registrador de destino vem da ALU (para um instrução do tipo R) ou da memória (para um load)

Instruções lógicas e aritméticas

- Executa a operação (opcode e funct) utilizando como operandos os valores em rs e rt
- Armazenam o resultado no banco de registradores (no endereço rd)
- Todas as instruções do tipo R precisam
  - Ler dois registradores
  - Realizar uma operação na UAL com os conteúdos dos registradores
  - Escrever o resultado em um registrador
- Instruções aritméticas lógicas: ADD, SUB, AND, OR
  - Ex: `add $t1, $t2, $t3`

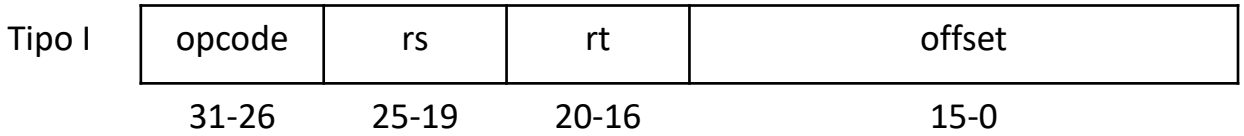


Instruções lógicas e aritméticas

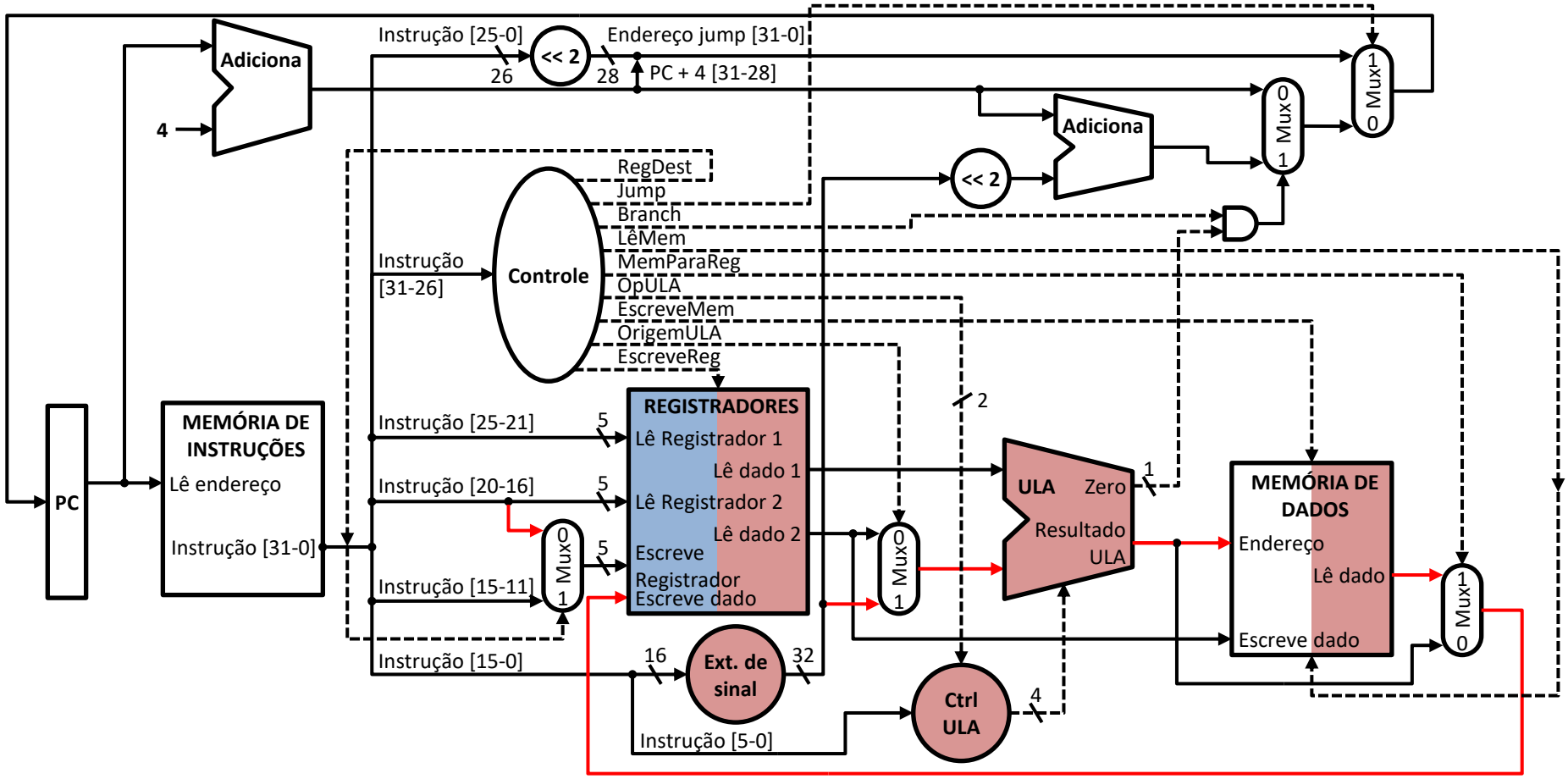


Instruções de load e store

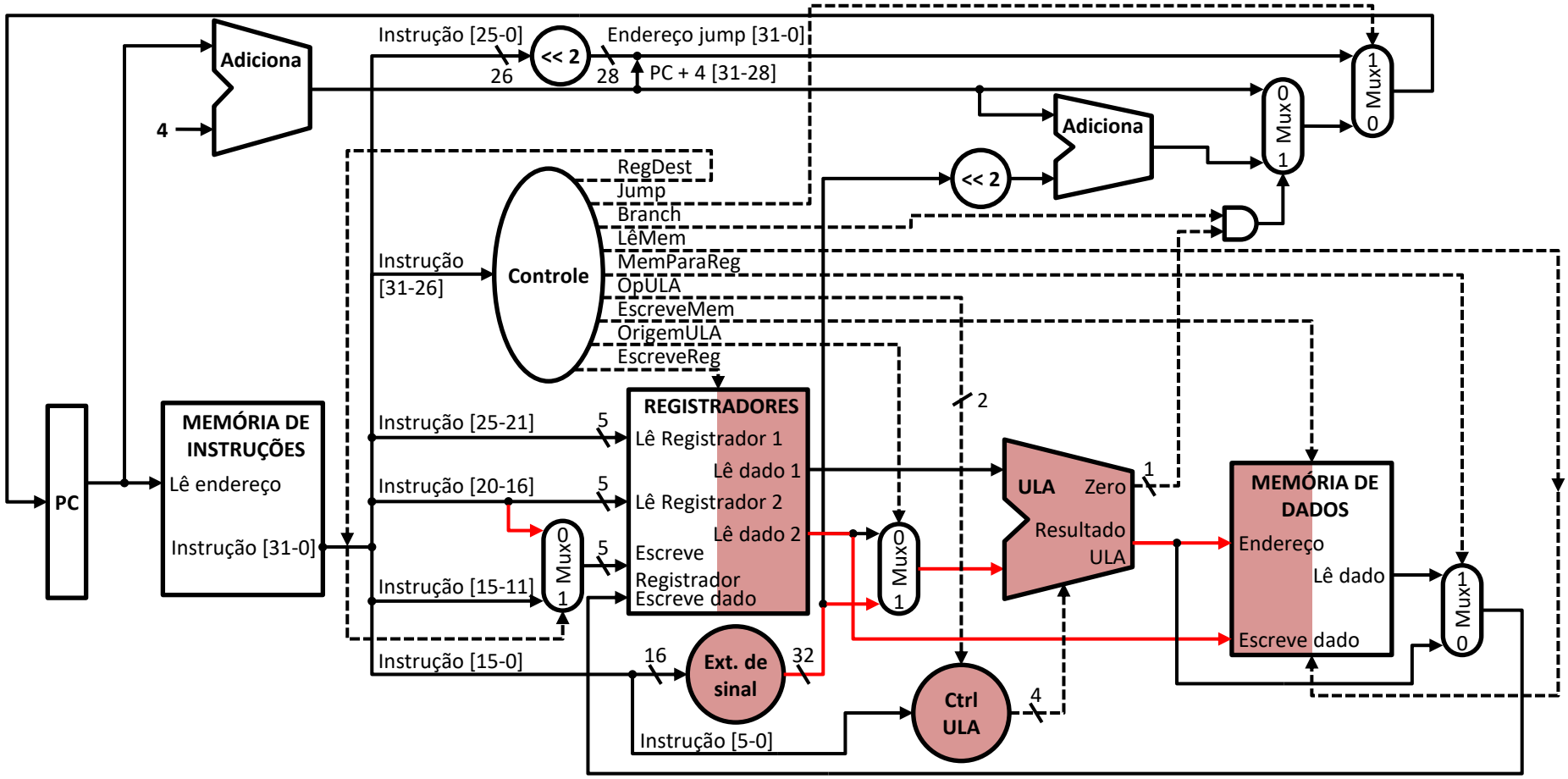
- `lw $t1, offset($t2)`
- `sw $t1, offset($t2)`
- Endereço de memória é calculado somando o registrador base (`$t2` no exemplo) ao número de 16 bits sem sinal estendido
  - `sw`: o valor a ser armazenado na memória de dados é lido do registrador.
  - `lw`: o valor é lido da memória de dados e escrito no registrador
- É necessária uma unidade para estender o sinal de 16 para 32 bits e uma memória para ler e escrever os dados.



Instrução de load



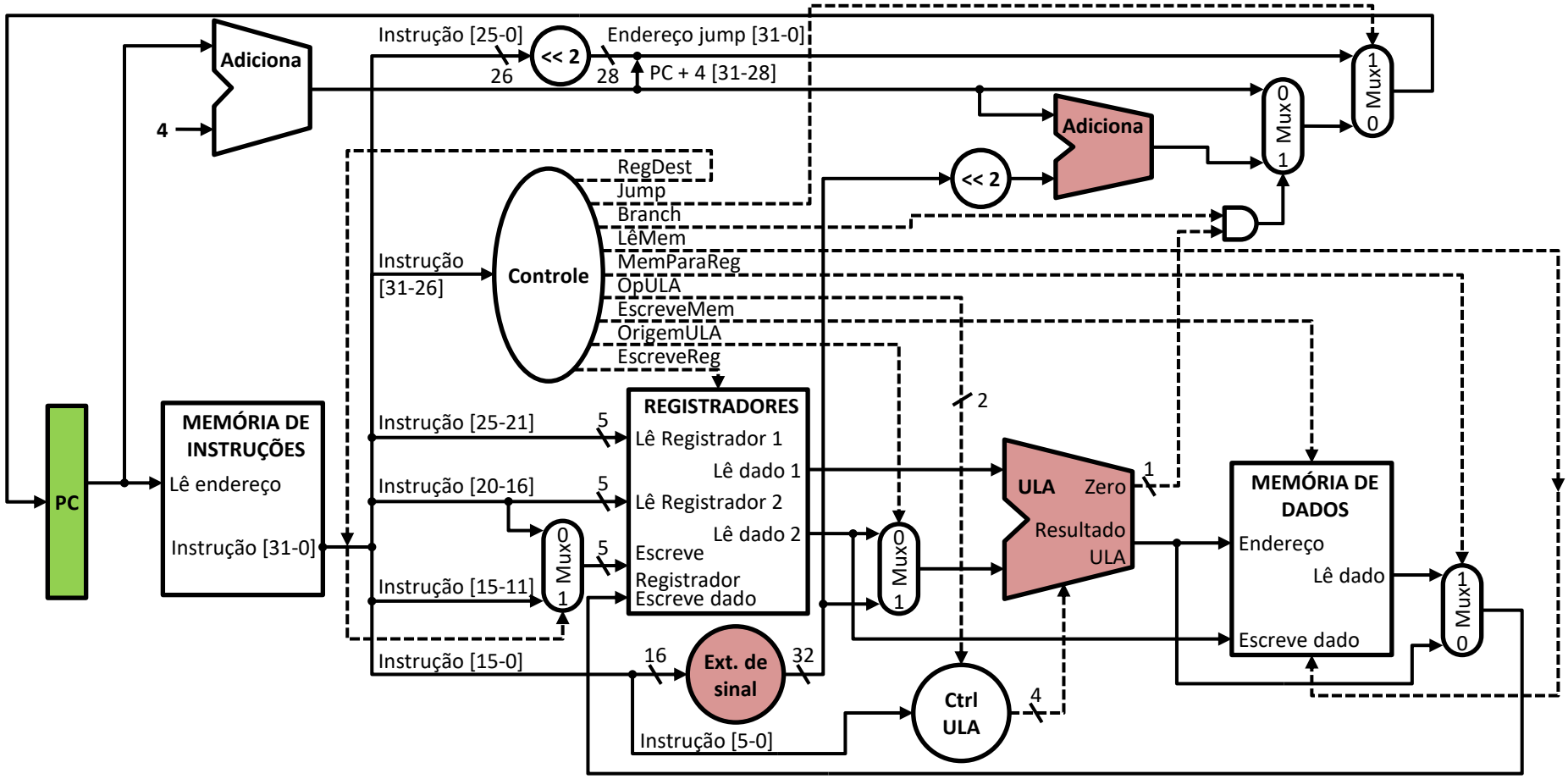
Instrução de store



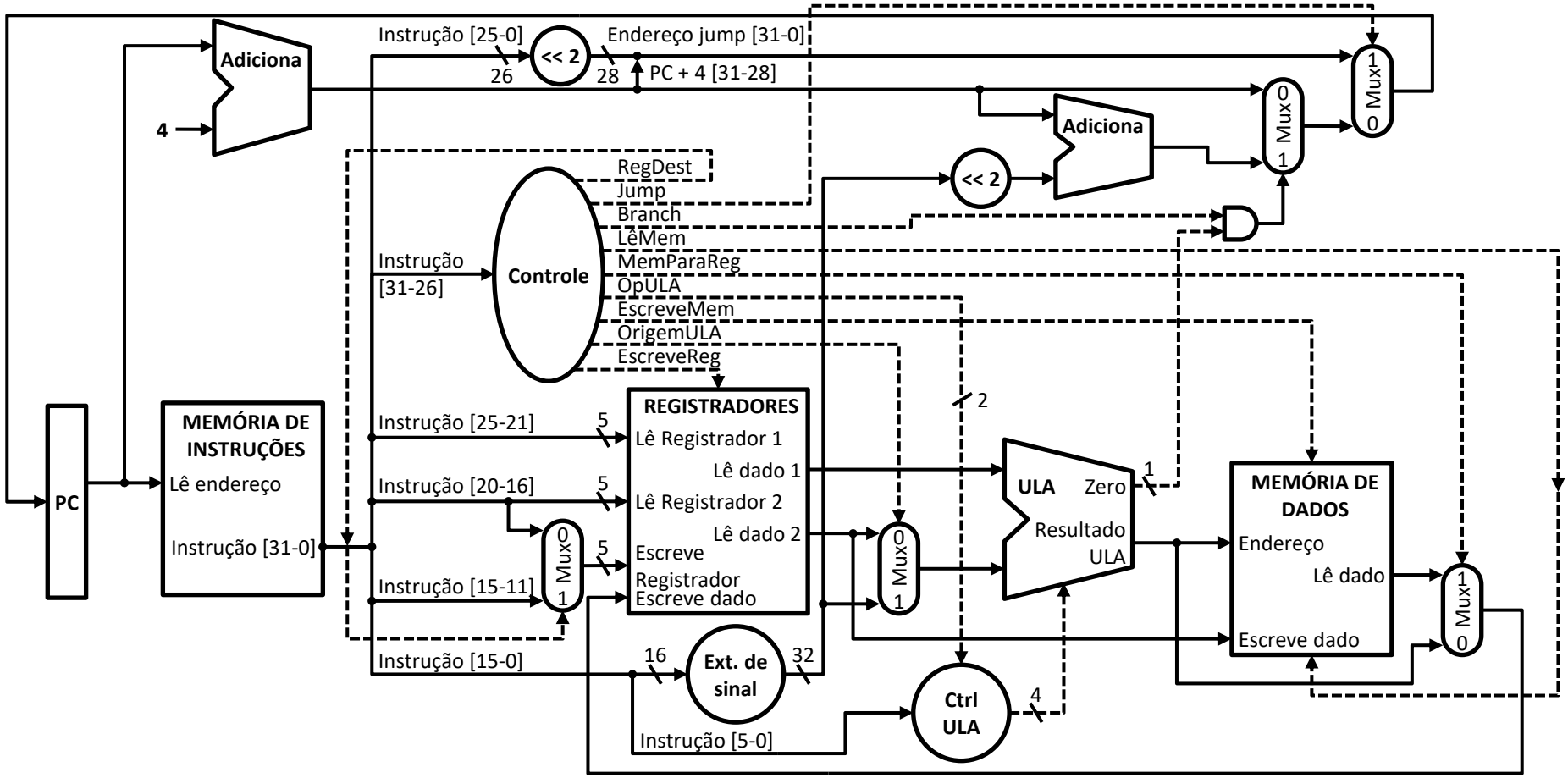
A instrução beq (bne)

- Possui três operandos
  - Dois registradores utilizados para a comparação
  - Offset indicando o endereço de memória para o deslocamento (PC + 4 + offset)
- Instruções de desvio
  - A base para o cálculo do endereço de desvio é o endereço da instrução seguinte ao desvio.
    - Como calculamos PC + 4 no caminho de dados para a busca de instruções, é fácil usar esse valor como a base para calcular o endereço de destino do desvio.
  - O campo offset é deslocado 2 bits para a esquerda de modo que resulte em uma offset de uma palavra;
    - Aumenta a amplitude do salto por um fator igual a 4
- Além de calcular o endereço do desvio, é necessário verificar se o desvio deve ser executado ou não, de acordo com a comparação entre os dois registradores
- Assim, o caminho de dados do desvio precisa de duas operações:
  - Calcular o endereço de destino do desvio
  - Comparar o conteúdo do registrador (sinal zero da UAL)

A instrução beq (bne)



Integrando o caminho de dados – MIPS Monociclo



Bibliografia

- PATTERSON, D. A.; HENNESSY, J. L. **Organização e Arquitetura de computadores: a interface hardware/software**. 3. ed. Rio de Janeiro: Elsevier, 2005.
  - CAPÍTULO 5
- Notas de aula da Profa. Mary Jane Irwin
  - CSE 331 Computer Organization and Design
  - http://www.cse.psu.edu/research/mdl/mji/mjicourses

Simuladores

- **MIPS-CPU:**
  - https://github.com/yuxincs/MIPS-CPU
  - MIPS monociclo e MIPS pipeline
  - Funciona no Logisim
    - http://www.cburch.com/logisim/pt/index.html
- **Mips-Datapath-Simulator:**
  - https://github.com/saliherdemk/Mips-Datapath-Simulator?tab=readme-ov-file
  - MIPS monociclo
  - Live demo:
    - https://saliherdemk.github.io/Mips-Datapath-Simulator/

Simuladores

- **WebMIPS:**
  - https://webmips.diism.unisi.it/
  - MIPS monociclo
  - Online
- **MIPS-Datapath:**
  - https://mi.eng.cam.ac.uk/~ahg/MIPS-Datapath/
  - MIPS monociclo
- **MIPS 101:**
  - https://www3.ntu.edu.sg/home/smitha/fyp_gerald/index.html
- **DrMIPS:**
  - https://brunonova.github.io/drmips/

FIM